Junior Java Backend fejlesztő II.

Java alapok II.

(tömbök, alprogramok)

Botlik Béla

Tömb I.

- A tömb egy összetett, referenciatípusú adatszerkezet, melyet azonos típusú, meghatározott számú elemek tárolására használunk.
- Az egyes elemekre indexen keresztül hivatkozunk.
- Egy tömb elemszámát a length adattaggal kérdezhetjük le.
  - Például → int elemSzam = <tömb neve>.length;
- Típusai:
  - Egydimenziós (vektor)
  - Többdimenziós (mátrix)

Tömb II. (vektor)

- Deklarálás:
  - <típus>[] <név>; // példa → int[] szamok;
- Inicializálás:
  - <név> = new <típus>[<elemszám>]; // példa → szamok = new int[3];
- Definiálás:
  - <név>[index] = <érték>; // példa → szamok[0] = 5;
- Egyidejű deklarálás és inicializálás:
  - <típus>[] <név> = new <típus>[<elemszám>]; // példa → int[] szamok = new int[3];
- Egyidejű inicializálás és definiálás:
  - <név> = new <típus>[] {<elem0>, …, <elemn>} // példa → szamok = new int[]{5,17,4};
- Egyidejű deklarálás, inicializálás és definiálás:
  - <típus>[] <név> = new <típus>[]{<elem0>,…,<elemn>}; //példa → int[] szamok = new int[]{5,17,4};→ int[] szamok = {5,17,4};

Tömb III. (mátrix)

- Mátrix esetén a szögletes zárójelek száma jelzi a dimenziók számát:
  - int[][] szamMatrix = new int[3][2]; // kétdimenziós, 3x2-es elemszámú tömb
- A másodlagos tömbök mérete eltérő is lehet:
  - int[][] szamMatrix = new int[3][];
  - szamMatrix[0] = new int[3];
  - szamMatrix[1] = new int[4];
  - szamMatrix[2] = new int[2];
    - // 3 elemű tömb, ami másodlagos tömböket tartalmaz: 3,4,2 elemszámmal.

Feladat I.

- A., Kérje be egy tanuló hat tantárgyának év végi eredményét.
  - Az adatokat tömbben tárolja.
  - Listázza ki egymás mellé az egyes osztályzatokat (szóközzel elválasztva).
  - Írja ki a tanuló átlagát.
- B., Oldja meg a feladatot -ahol lehet- bővített for ciklussal is. (foreach szerkezet)

Feladat II.

- A., Hozzon létre egy 5x3-as kétdimenziós tömböt és töltse fel 1-től kezdődő futó sorszámmal.
  - Listázza ki a tömb elemeit az elsőtől az utolsóig.
  - Listázza ki a tömb elemeit az utolsótól az elsőig.
- B., Debugger használatával lépkedjen végig a feltöltő és kiíró algoritmus utasításain, a működés könnyebb megértése végett.
  - Használjon töréspontot.
  - Figyelje az egyes értékek változásait.
- C., Oldja meg a sorszámozást segédváltozó bevezetése nélkül.

Önálló (házi) feladat I.

- Töltsünk fel egy 10 elemű tömböt a felhasználótól bekért egész számokkal.
  - A., Írjuk ki az adatokat egymás mellé szóközzel elválasztva!
  - B., Írjuk ki az első elem négyzetét!
  - C., Írjuk ki a páratlan számokat és indexeiket (soronként egy szám-index párt kiírva)!
  - D., Írjuk ki a hárommal osztható számokat a tömb utolsó elemétől az első felé haladva!
  - E., Írjuk ki az első öt elem összegét és külön az utolsó öt elem összegét!
  - F., Hozzunk létre egy ugyancsak 10 elemű, de kétdimenziós tömböt és töltsük fel ugyanazokkal az értékekkel mint az első tömböt!
  - G., Listázzuk ki egymás alá a számokat úgy, hogy minden szám mellé írjuk ki mindkét tömbbeli indexét!

Karakterfüggvények

- Karakter típusú változókon műveleteket végrehajtó metódusok:
  - Logikai eredményt szolgáltató metódusok:
    - Character.isLetter(c) - visszatérési érték igaz, ha a karakter betű
    - Character.isLowerCase(c) - visszatérési érték igaz, ha a karakter kisbetű
    - Character.isUpperCase(c) - visszatérési érték igaz, ha a karakter nagybetű
    - Character.isDigit(c) - visszatérési érték igaz, ha a karakter számjegy
  - Konvertáló és összehasonlító metódusok:
    - Character.toUpperCase(c) - a paraméterben átadott karakter nagybetűs alakjával tér vissza
    - Character.toLowerCase(c) - az átadott karakter kisbetűs alakjával tér vissza
    - Character.compare(c1, c2) - a paraméterekként átadott karakterek összehasonlítását végzi

Sztringfüggvények I.

- Sztring típusú változókon műveleteket végrehajtó metódusok:
  - <string>.length() - a sztring hosszával tér vissza
  - <string>. toUpper() - nagybetűs szöveggé alakít
  - <string>.toLower() - kisbetűs szöveggé alakít
  - <string>.substring(index,index2) - részsztring képzés(honnan,meddig)
  - <string>.replace(”szöveg”,”új szöveg”) - kicserél a szövegben bizonyos részeket egy magadott új részre.
  - <string>.split(”elválasztó”) - feldarabolja a szöveget bizonyos tördelési pontoknál és String[] tömbbel tér vissza.
  - <string>.contains(”szöveg”) - teszteli, hogy a string tartalmazza-e a megadott szöveget.

Sztringfüggvények II.

- További sztringfüggvények:
  - <string>.startsWith(”szöveg”) - teszteli, hogy a string a megadott szöveggel kezdődik-e.
  - <string>.endsWith(”szöveg”) - teszteli, hogy a string a megadott szöveggel végződik-e.
  - <string>.indexOf(”szöveg”) - megadja az adott szöveg első előfordulását a stringben.
  - <string>.lastIndexOf(”szöveg”) - megadja az adott szöveg utolsó előfordulását a stringben.
  - <string>.trim() - szóközök levágása a szöveg elejéről és végéről.
  - <string>.equals(”szöveg”) - teszteli, hogy a két szöveg megegyezik-e.

Matematikai függvények

- A Math osztály matematikai műveletek és konstansok elérését biztosítja:
  - Használata → Math.<metódus() vagy konstans>;
  - **Függvények:**
    - abs(érték) - abszolút értékkel tér vissza
    - min(érték1, érték2) - két szám közül a kisebbel tér vissza
    - max(érték1, érték2)- két szám közül a nagyobbal tér vissza
    - sqrt(érték) - a szám négyzetgyökével tér vissza
    - pow(alap, kitevő) - az alap hatványkitevőre emelésével tér vissza
    - round(érték) - a legközelebbi egész számra kerekít
    - floor(érték) - lefelé kerekítés
    - ceil(érték) - felfelé kerekítés

Konstansok:

Pi (3.141592653589793)

E (2.718281828459045)

Dátumfüggvények

- Dátum -és idő- tárolására és kezelésére több lehetőség kínálkozik, az alábbi osztályok használatával:
  - Date (java.util package)
  - Calendar (java.util)
  - LocalDate (java.time)
  - LocalTime (java.time)
  - LocalDateTime (java.time)

Feladat III.

- Kérjük be a konzolon a felhasználó vezetéknevét. Majd kérjük be a keresztnevét is. Képezzünk számára csupa kisbetűs felhasználói azonosítót, ami a vezetéknév első 3 betűje, hozzáfűzve a keresztnév első 3 betűje, kiegészítve egy számmal, ami legyen a teljes névben található ‚e’ betűk száma (függetlenül, hogy kis vagy nagybetű). Írjuk ki a képernyőre a felhasználó teljes nevét, és a generált azonosítóját.
- Példa: Straub Ingrid → string0

Véletlenszám generátor

- Adott intervallumon belül számokat véletlenszerűen előállító algoritmus.
- Alkalmazási területei:
  - Szoftvertesztelés
  - Játékprogramozás
- Használata:
  - Random <változónév> = new Random();
    - Random osztály használatához szükség van a java.util package importálására.
  - <változónév>.nextInt(<maximum érték>);
    - Java17 óta .nextInt(<minimum érték>, <maximum érték>)
  - <változónév>.nextDouble();

Feladat IV.

- Hozzon létre két egydimenziós tömböt, elemszámuk legyen véletlenszerű (5 és 10 közötti). Töltse fel mindkettőt 1 és 50 közötti véletlenszámokkal.
  - Listázza ki tömbönként:
    - az összes elemet
    - az elemek összegét
    - a páros számok darabszámát
    - a legnagyobb elemet
    - az első négyzetszámot

Önálló (házi) feladat II.

- A., Kérjünk be a felhasználótól tíz keresztnevet.
  - A neveket nagybetűs formátumban, vektorban tároljuk!
  - Listázzuk ki a tömb minden második elemét!
  - Állapítsuk meg, melyik a leghosszabb név!
  - Írjuk ki, melyik névben van a legtöbb ’s’ betű!
- B., Töltsünk fel egy 4x3-as mátrixot 50 és 100 közötti véletlenszámokkal.
  - Transzponáljuk egy másik mátrixba!
  - Listázzuk ki mindkét mátrixot a megfelelő sor/oszlop formátumban!
- C., Töltsünk fel egy 3x3-as mátrixot egyjegyű véletlenszámok négyzetre emelt értékeivel.
  - Listázzuk ki a főátlóban található számokat és összegüket!
  - Listázzuk ki fordított sorrendben (az utolsó elemtől az elsőig) az eredetileg generált számokat!

Metódusok I.

- Ismétlődő utasításblokkok egybeszervezése.
- Előnyei:
  - Jobban áttekinthető, és rövidebb forráskód
  - Könnyebb módosíthatóság
  - Más osztályok metódusait is elérhetjük
- Típusai:
  - Visszatérési értékkel rendelkező
  - Visszatérési érték nélküli

Metódusok II.

- Visszatérési érték nélküli metódus deklarációja:
  - void <név>(<paraméterek>){

    // metódustörzs
    }
- Példa:
  - void osszeadasKiir(int a, int b){

    System.out.println(a + b);
    }

Metódusok III.

- Visszatérési értékkel rendelkező metódus deklarációja:
  - <típus> <név>(<paraméterek>){

    // metódustörzs
    }
- Példa:
  - int osszeadas(int a, int b){

    return a + b;
    }

Metódusok IV.

- Metódus hívását az alprogram nevére hivatkozva tehetjük meg:
  - Például:
    - <metódusnév>();
    - <metódusnév>(<paraméterek>);
    - <változónév> = <metódusnév>(<paraméterek>);
    - <metódusnév>(<metódusnév>(<paraméterek>));
- Paraméterezés
  - A híváskor átadott paraméterek darabszáma, típusa és sorrendje egyezőséget kell mutasson a metódus deklarációjával.
  - A paraméterek érték szerint kerülnek átadásra.

Tiszta kód III.

- Kerüljük az ismétlődéseket, a közös utasításokat emeljük ki metódusba.
- A jó metódus elárulja rendeltetését, és lehetőleg csak egy feladatot hajt végre
  - a hossza pedig néhány soros (de ne legyen több 20 sornál)
  - jobbára egy elágazásból vagy ciklusból áll, melynek blokkjában egy utasítás (vagy újabb alprogram hívás) található.
  - minél rövidebb egy metódus, annál könnyebb beszédes elnevezést találni neki.
- A metódusok minél kevesebb paramétert várjanak (ideálisan egyet se)
  - a működés könnyebben értelmezhető
  - a tesztelés hatványozottan nehezedik a paraméterek számának növekedésével
  - logikai változót (jelzőparaméter) ne adjunk át paraméterként!

Feladat V.

- Optimalizálja feladat IV. megoldását metódusok használatával úgy, hogy a belépési main függvény csak deklarációkat és metódushívásokat tartalmazhat.

Önálló (házi) feladat III.

- Készítsen alkalmazást, ami vállalkozások nyereségét/veszteségét tárolja, havi bontásban. Használjon metódusokat a megoldáshoz!
  - Kérje be a felhasználótól a vállalkozások számát, majd ennek megfelelően minden vállalkozás előző évi eredményét havi bontásban. Tárolja az adatokat vállalkozásonként a megfelelő adatszerkezetben!
  - Listázza ki a vállalkozások havi eredményeit és éves összesítésüket!
  - Generáljon egy véletlenszámot (1-12 között), és csak annak a hónapnak az eredményeit írja ki!
  - Állapítsa meg, melyik vállalkozás érte el a legjobb éves eredményt!
  - Írja ki vállalkozásonként a veszteséges hónapok számát!

Alprogram túltöltés I.

- Alprogram túltöltés/túlterhelés (method overloading) alatt az azonos nevű, de eltérő paraméterezéssel rendelkező metódusokat értjük.
- A fordító a metódus szignatúrája alapján dönti el, melyik alprogramot kell futtatnia.
  - metódus szignatúra = metódus neve + paraméterlistája
  - A fordító a minél pontosabb típusegyezést keresi.

Alprogram túltöltés II.

- Megvalósítása:
  - udvozlo() {
    System.out.println(„Üdvözlöm!”);
    }
  - udvozlo(String nev) {
    System.out.println(„Üdvözlöm! ”+nev);
    }

Feladat VI.

- A., Készítsen programot, mely ki tudja számolni egy téglalap, négyzet és háromszög kerületét. A megoldáshoz használjon alprogram túltöltést.
- B., Bővítse a programot a kör kerületének kiszámolási lehetőségével!
- C., Készítse el túltöltéses metódusokkal a területszámítást is!
- Képletek:
  - Téglalap: K: 2*(a+b); T: a*b
  - Négyzet: K: 4*a; T: a^2
  - Háromszög: K: a+b+c; T: (a*ma)/2
  - Kör: K: 2*r*π; T: r^2*π

Feladat VII - rendezések.

- A.,Töltsön fel egy negyven elemű tömböt 1-100 közötti véletlenszámokkal úgy, hogy egy szám csak egyszer szerepelhet a tömbben.
  - Listázza ki az elemeket 5x8-as formátumban.
  - Rendezze a tömböt csökkenő sorrendbe.
  - Listázza ki a rendezett tömböt 4x10-es formátumban.
  - Kérjen be a felhasználótól egy számot, majd bináris kereséssel állapítsa meg, a szám megtalálható-e a tömbben.
- B., Egy tömb elemei: {Budapest, Debrecen, Szeged, Miskolc, Pécs, Győr, Nyíregyháza, Kecskemét, Székesfehérvár, Szombathely }
  - Rendezze a tömbelemeket ABC szerint növekvő sorrendbe és listázza ki őket.

A két feladathoz különböző rendezési algoritmust használjon!

Alapszintű fájlkezelés

- Java adatfolyamként (Stream) kezeli a fájlokat.
- Folyamat:
  - adatfolyam megnyitása
  - amíg van újabb információ
    - addig olvasás / írás
  - adatfolyam bezárása
- Az adatfolyam bezárása kiemelten fontos, adatvesztés elkerülése végett.
- A fájlműveletek végző osztályok az io (nio) csomagban találhatók.

Feladat VIII. - fájlkezelés

- Egy osztály névsorának tárolását kell megoldanunk. Készítsünk egy menüt, ahol a felhasználó a következő lehetőségek közül választhat:
  - 1. Adatok bekérése konzolról
  - 2. Adatok beolvasása fájlból
  - 3. Névsor kilistázása
  - 4. Adatok kiírása fájlba
  - 5. Kilépés
- Hajtsuk végre a választott feladatot. A menü minden feladat után térjen vissza mindaddig, amíg a felhasználó kilépést nem választ.